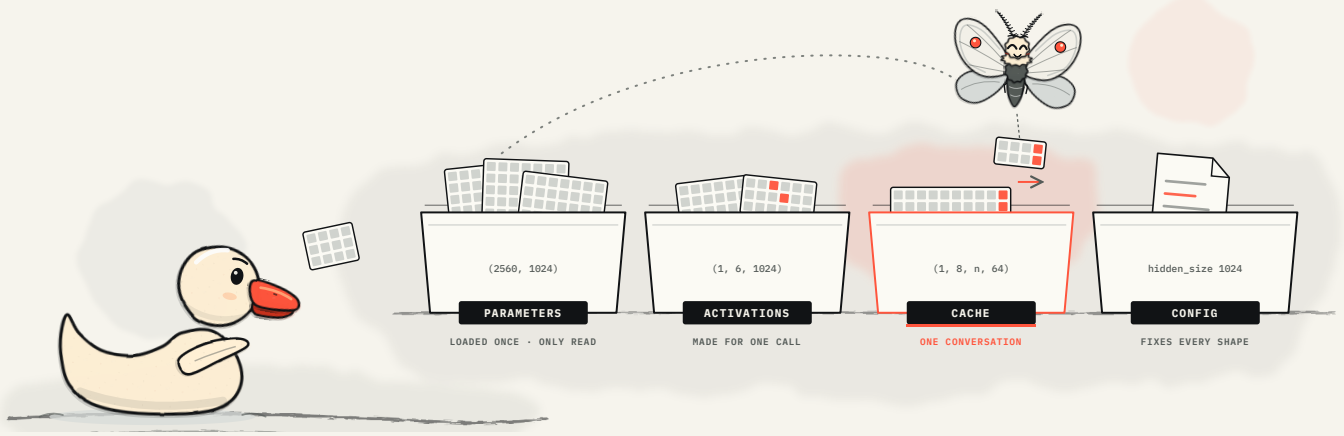


B1C02 · CHAPTER 2

Tensors, Weights, Layers, and State

WATCH · 1:05

Four kinds of arrays

IN THIS CHAPTER

What you will be able to do

1. Name every axis of a real shape, pick the slice you mean, and spot the wrong-axis bug from its shape.
2. Read a parameter's name and shape: which block, which part, how many numbers go in and come out, and which config values set that shape.
3. Count the parameters once, even when a shared array appears under two names.
4. Tell parameters from activations, and show that running a model never changes its parameters.
5. Explain what the cache keeps between steps, how it grows, and why it belongs to exactly one conversation.
6. Sort any array in a running model by kind and lifetime, and name the chapter that covers each part in depth.

Chapter 1 ran a small open model, LiquidAI's LFM2.5-230M, on a laptop CPU, and printed three shapes on the way: `(1, 6)`, `(1, 6, 1024)`, and `(1, 6, 65536)`. This chapter explains what each number in those shapes means. Then it sorts every array a running model touches into one of four kinds, because the kinds behave very differently.

TERM **tensor**
An array of numbers with a shape. Each position in the shape is an axis.

TERM **shape**
The list of axis sizes, in order. (2, 6, 1024) has three axes.

TERM **axis**
One direction you can index along, like batch, position, or width. Name it before you use it.

TERM **Parameters**
The model's learned numbers, also called weights. Training sets them. Running the model only reads them.

TERM **Activations**
An array the model computes during one call, like a block's output. Made fresh each call, then dropped.

WATCH · 1:09

Name every axis

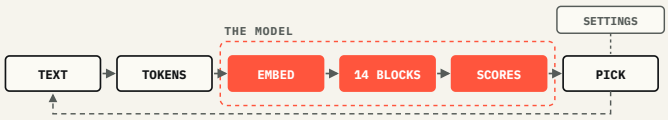
Everything numeric inside a model is an array of numbers with a shape. Such an array is called a **tensor**. The **shape** is the list of its sizes, in order. Each position in the shape is an **axis**, one direction you can index along. `(1, 6, 1024)` has three axes, of sizes 1, 6, and 1,024.

Here are the four kinds of arrays:

- **Parameters** are the model's learned numbers. They are loaded once and only read.
- **Activations** are arrays the model computes during one call. They are made fresh each call, then dropped.
- **The cache** is what the model keeps between steps of one reply, so it does not redo earlier work. It belongs to one conversation.
- **The config** is a small file of numbers, like the width and the number of blocks. It is not an array, but it fixes every array's shape.

Mixing up the kinds causes real bugs: a slice that answers the wrong question, a parameter count that is too big, and an answer about the wrong country. We run each case on the same model as chapter 1, with the real output.

2.1 Every value has a shape



The scripts in this chapter import the same `load.py` as chapter 1. It loads the pinned revision of the model in Transformers:

LISTING 2-1 load.py

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

REPO = "LiquidAI/LFM2.5-230M"
REV = "40cb2ad3b3044d5a41eee083a6103c8b523afa45"
tok = AutoTokenizer.from_pretrained(REPO, revision=REV)
model = AutoModelForCausalLM.from_pretrained(
    REPO, revision=REV, dtype=torch.bfloat16).eval()
```



TECHNICAL STUFF

Chapter 1 always sent one prompt, so every shape started with 1. Send two prompts in one call, and that 1 changes:

LISTING 2-2 shapes.py

```
import torch
from load import tok, model

texts = ["The capital of France is",
         "The capital of Spain is"]
ids = tok(texts, return_tensors="pt").input_ids
with torch.no_grad():
    out = model(ids, output_hidden_states=True)
emb = out.hidden_states[0]           # the embedding output
print("token IDs ", tuple(ids.shape))
print("embeddings", tuple(emb.shape))
print("logits    ", tuple(out.logits.shape))
for b, text in enumerate(texts):
    top = int(out.logits[b, -1].argmax())  # prompt b, last position
    print(f"{text!r:28} -> {tok.decode([top])!r}")
```



TECHNICAL STUFF

OUTPUT OF LISTING 2-2

```
$ python3 shapes.py
token IDs  (2, 6)
embeddings (2, 6, 1024)
logits     (2, 6, 65536)
'The capital of France is'   -> ' Paris'
'The capital of Spain is'    -> ' Madrid'
```

Read each shape from left to right, and name each axis as you go:

ARRAY	SHAPE	AXES, IN ORDER
token IDs	(2, 6)	batch, position
embeddings	(2, 6, 1024)	batch, position, width
logits	(2, 6, 65536)	batch, position, vocabulary

TERM **batch**
Several independent prompts stacked into one array, so one call handles them all.

TERM **position**
Where a token sits in the sequence: 0 for the first token, 1 for the next.

TERM **width**
How many numbers describe one token inside the model: 1,024 here.

The first axis is the **batch**: several independent prompts stacked into one array, so one call handles them all. Here it has two entries, France and Spain. The second axis is the **position**, where a token sits in the sequence. Both prompts are six tokens long, so it has six entries. The embeddings add a third axis, the **width**: how many numbers describe one token inside the model. That is 1,024 here. The logits replace the width with the vocabulary: one score for each of the 65,536 possible next tokens.

The two prompts do not affect each other. Prompt 0 gets `Paris` at 95.22 percent, exactly as it did alone in chapter 1. Prompt 1 gets `Madrid` at 93.66 percent. Batching is a way to do two calls at once, not a way to mix them.

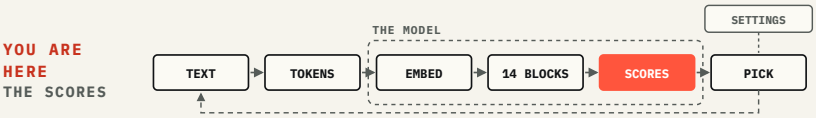
To get one prompt's next token, the script indexes `out.logits[b, -1]`. That means batch entry `b`, last position, every vocabulary entry. An axis you give an index to drops out. An axis you leave out keeps all its entries. The result has one axis left, of size 65,536, and `argmax` picks the highest score on it.



QUICK CHECK

CHECK
The batch went from one prompt to two. Which axis of the logits changed, and which stayed? Only the first axis changed, from 1 to 2. The position axis is still 6, and the vocabulary axis is still 65,536.

2.2 The wrong axis



WATCH · 1:06

The wrong axis

Indexing code runs as long as the indexes fit the shape. It does not check that you meant the right axis. Here are two ways to ask for "the last one":



TECHNICAL STUFF

LISTING 2-3 wrong.py

```
import torch
from load import tok, model

texts = ["The capital of France is",
         "The capital of Spain is"]
ids = tok(texts, return_tensors="pt").input_ids
with torch.no_grad():
    logits = model(ids).logits
    ↳ # (batch, position, vocabulary)
right = logits[:, -1]   # every prompt, last position
wrong = logits[-1]      # last prompt, every position
print("right", tuple(right.shape))
print([tok.decode([int(i)]) for i in right.argmax(-1)])
print("wrong", tuple(wrong.shape))
print([tok.decode([int(i)]) for i in wrong.argmax(-1)])
```

OUTPUT OF LISTING 2-3

```
$ python3 wrong.py
right (2, 65536)
[' Paris', ' Madrid']
wrong (6, 65536)
['<|startoftext|>', ' ', ' of', ' France', ' is', ' Madrid']
```

`logits[:, -1]` keeps every batch entry and takes the last position. That gives one next token per prompt: `Paris` and `Madrid`. `logits[-1]` indexes the first axis instead. It takes the last batch

entry, the Spain prompt, and keeps all six positions. That gives six predictions, one after each position of the Spain prompt.

Both lines run without an error, and both print real tokens. The wrong one even looks plausible. Its fourth entry is the model's guess after `The capital of`, and the guess is `France`, at 19 percent. A bug like this can slip through a quick glance at the output.



The shape catches it. You wanted one row per prompt, so the first size should be 2. It is 6. When you index an array, write down the shape you expect, then compare it to the shape you got.



QUICK CHECK

CHECK

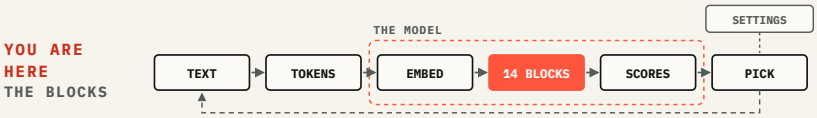
A function expects `(batch, vocabulary)` and receives `(6, 65536)` from a batch of two prompts. What went wrong? The code indexed the batch axis where it meant the position axis. It got every position of one prompt instead of the last position of every prompt.

WATCH · 1:21



Read a weight's name

2.3 Reading a parameter's name



Chapter 1 counted 229,693,184 parameters. They live in named arrays. The script below lists them all, then prints the names and shapes for everything outside the blocks, plus one conv block and one attention block:



TECHNICAL STUFF

LISTING 2-4 names.py

```
from load import model

params = list(model.named_parameters())
print(len(params), "tensors")
for name, p in params:
    block = name.split(".")[2] if ".layers." in name else ""
    if block in ("", "0", "2"):
    ↳ # one conv block, one attention block
        print(f"{name.removeprefix('model.'):38} {tuple(p.shape)}")
```

OUTPUT OF LISTING 2-4

```
$ python3 names.py
132 tensors
embed_tokens.weight                    (65536, 1024)
layers.0.conv.conv.weight              (1024, 1, 3)
layers.0.conv.in_proj.weight           (3072, 1024)
layers.0.conv.out_proj.weight          (1024, 1024)
layers.0.feed_forward.w1.weight        (2560, 1024)
layers.0.feed_forward.w3.weight        (2560, 1024)
layers.0.feed_forward.w2.weight        (1024, 2560)
layers.0.operator_norm.weight          (1024,)
layers.0.ffn_norm.weight               (1024,)
layers.2.self_attn.q_proj.weight       (1024, 1024)
layers.2.self_attn.k_proj.weight       (512, 1024)
layers.2.self_attn.v_proj.weight       (512, 1024)
layers.2.self_attn.out_proj.weight     (1024, 1024)
layers.2.self_attn.q_layernorm.weight  (64,)
layers.2.self_attn.k_layernorm.weight  (64,)
layers.2.feed_forward.w1.weight        (2560, 1024)
layers.2.feed_forward.w3.weight        (2560, 1024)
layers.2.feed_forward.w2.weight        (1024, 2560)
layers.2.operator_norm.weight          (1024,)
layers.2.ffn_norm.weight               (1024,)
embedding_norm.weight                  (1024,)
```

The model's 229,693,184 numbers sit in 132 arrays. A name reads like a path, from the outside in. Take `layers.2.self_attn.q_proj.weight`:

- `layers.2` is block 2, the third block. The code calls the blocks `layers`.

- `self_attn` is that block's mixer, an attention mixer.
- `q_proj` is one part of the mixer, the query projection. Chapter 8 explains what it does.
- `weight` is the array of learned numbers that part uses.

TERM **block**
One repeated stage that reworks every token's numbers. This model stacks 14 of them. The code calls them layers.

The word "layer" means different things in different places. In this code, `layers.2` is a whole **block**, the repeated stage from chapter 1. Each block holds 8 or 11 arrays. People also call a single matrix inside a block a layer. So when someone says "layer," read the name to see which they mean.

The names show the two kinds of block. Block 0 is a conv block. Its mixer has a `conv` filter and two projections. Block 2 is an attention block. Its mixer has query, key, value, and output projections, plus two small norms. Both have the same feed-forward part, `w1`, `w3`, and `w2`, and the same two norms. All eight conv blocks hold the same eight array names, and all six attention blocks hold the same eleven. One name misleads. `embedding_norm.weight` is the final norm: the model applies it after the last block, not after the embeddings. A name is a strong hint, and the model code is the check. With the embedding table and the final norm, that makes 1 + 8 × 8 + 6 × 11 + 1 = 132 arrays.

TERM **weight matrix**
A parameter with two axes, stored as (numbers out, numbers in).

Most of these arrays have two axes. An array with two axes is a **weight matrix**, and this code stores it as (numbers out, numbers in). `w1` has shape (2560, 1024), so it takes a token's 1,024 numbers and produces 2,560. `w2` has shape (1024, 2560), so it brings them back to 1,024. That is the widening and narrowing of the feed-forward part from chapter 1. The embedding table also has two axes, but the model reads it as a lookup: each token ID selects one row. The norms are single lists of 1,024 numbers, one scale per number in a token's vector. Chapter 3 shows what multiplying by a weight matrix computes.

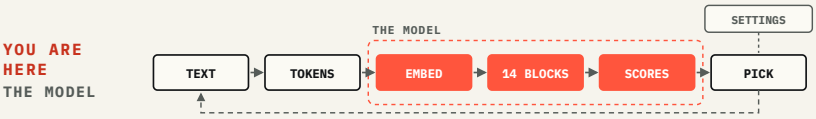
CHECK

What does `layers.4.self_attn.k_proj.weight` hold, and what is its shape? It is the key projection in block 4's attention mixer. Every attention block has the same arrays, so its shape is (512, 1024), like block 2's.



QUICK CHECK

2.4 The config sets every shape



WATCH · 1:13

The config sets every shape

The shapes are not arbitrary. A few numbers in the model's **config** file, `config.json`, fix all of them:

TERM config

The small file of numbers, like the width and the number of blocks, that fixes every array's shape.

LISTING 2-5 config.py

```
from load import model

c = model.config
for k in ("vocab_size", "hidden_size", "intermediate_size",
          "num_hidden_layers", "num_attention_heads",
          "num_key_value_heads", "conv_L_cache"):
    print(f"{k:20} {getattr(c, k)}")
```

OUTPUT OF LISTING 2-5

```
$ python3 config.py
vocab_size           65536
hidden_size          1024
intermediate_size    2560
num_hidden_layers    14
num_attention_heads  16
num_key_value_heads  8
conv_L_cache         3
```



TECHNICAL STUFF

Match each number to the shapes in `names.py`:

CONFIG VALUE	SHAPES IT SETS
`vocab_size` 65,536 and `hidden_size` 1,024	the embedding table (65536, 1024)
`hidden_size` 1,024	the width of every block's input and output, every norm, `out_proj` (1024, 1024)
`intermediate_size` 2,560	`w1` and `w3` (2560, 1024), `w2` (1024, 2560)
`num_attention_heads` 16	`q_proj` (1024, 1024): 16 heads of 64 numbers each, and 16 × 64 = 1,024
`num_key_value_heads` 8	`k_proj` and `v_proj` (512, 1024): 8 groups of 64, and 8 × 64 = 512
`conv_L_cache` 3	the conv filter (1024, 1, 3): one filter 3 positions wide for each of the 1,024 numbers
`num_hidden_layers` 14	14 blocks, `layers.0` to `layers.13`

The width of 64 per head is `hidden_size` divided by `num_attention_heads`. The conv block's `in_proj` makes three 1,024-wide parts at once, which gives 3,072. The code calls them B, C, and x. x holds the numbers the filter mixes, and B and C scale them on the way in and on the way out. The conv filter's middle 1 means each of the 1,024 filters reads only its own number. Chapter 10 opens the conv block.

The config is not learned, and it holds no arrays. The loading code reads the config, builds empty arrays of those shapes, and then fills them from the weights file. The learned values fit only the shapes they were trained in. We set `intermediate_size` to 3,072 and loaded the same weights, and loading stopped with a size-mismatch error, because the stored arrays no longer fit.



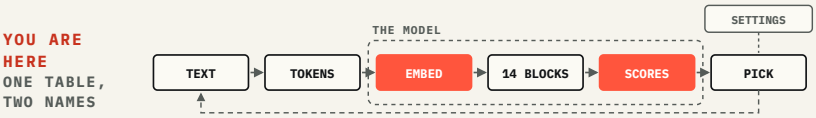
QUICK CHECK

CHECK

Which config value would you change to make the feed-forward part wider, and which three arrays would change shape?

`intermediate_size`. It sets the shapes of `w1`, `w3`, and `w2`. A wider model also needs new learned values, because the old ones do not fit.

2.5 Counting the parameters once



Code that saves or loads models works with the model's `state_dict()`, a mapping from each name to its array. Counting the numbers in it seems like a good way to count parameters. It gives the wrong answer:



TECHNICAL STUFF

LISTING 2-6 count.py

```
from load import model

params = sum(p.numel() for p in model.parameters())
saved = sum(t.numel() for t in model.state_dict().values())
print(f"parameters  {params:>12,}")
print(f"state_dict  {saved:>12,}")
sd = model.state_dict()
print("lm_head.weight listed:", "lm_head.weight" in sd)
print("same storage:", sd["lm_head.weight"].data_ptr()
      == sd["model.embed_tokens.weight"].data_ptr())
```

OUTPUT OF LISTING 2-6

```
$ python3 count.py
parameters   229,693,184
state_dict   296,802,048
lm_head.weight listed: True
same storage: True
```

The state dict counts 67,108,864 more numbers, exactly one embedding table of 65,536 × 1,024. Chapter 1 showed why. The output layer reuses the embedding table instead of holding its own. The state dict lists that one array under two names, `model.embed_tokens.weight` and `lm_head.weight`, and summing it



counts the table twice. `model.parameters()` gives each shared array once.

Equal values would not prove the two names are one array. Two separate copies could hold the same numbers. The last line checks the address in memory where each array's numbers start. Both names point to the same address, so they are the same array.



QUICK CHECK

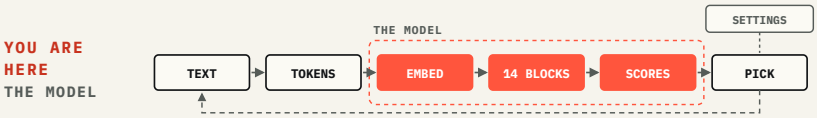
CHECK

A tool reports 296.8 million parameters for this model. What would you check first? Whether it summed every name in a saved listing. The extra 67.1 million is the embedding table counted a second time, under the output layer's name.

WATCH · 1:21

Made for one call

2.6 Activations: made for one call



The parameters are what the model holds. Everything it computes from them during a call is an **activation**: the embeddings, every block's output, the 2,560-wide numbers inside each feed-forward part, the logits. The script below hooks every layer that holds weights and adds up the numbers each one hands on. It runs a 6-token prompt and a 352-token prompt:



TECHNICAL STUFF

LISTING 2-7 activations.py

```
import torch
from load import tok, model

made = []                       # numbers each layer hands on
def hook(module, inputs, out):
    made.append(out.numel())
for m in model.modules():       # every layer that holds weights
    if list(m.parameters(recurse=False)):
```

CONTINUES OVERLEAF →

activations.py CONTINUED

```
        m.register_forward_hook(hook)

params = sum(p.numel() for p in model.parameters())
for text in ("The capital of France is",
             "The capital of France is Paris. " * 50):
    ids = tok(text, return_tensors="pt").input_ids
    made.clear()
    with torch.no_grad():
        model(ids)
    print(f"{ids.shape[1]:>4} IDs  activations {sum(made):>12,}"
          f"  parameters {params:,}")
```

OUTPUT OF LISTING 2-7

```
$ python3 activations.py
   6 IDs  activations    1,456,128  parameters 229,693,184
 352 IDs  activations   85,426,176  parameters 229,693,184
```

The parameters did not change with the prompt. The activations grew in step with it: 242,688 numbers per position, both times. Every activation has a position axis, so a longer prompt makes every one of them longer, and a bigger batch multiplies them again. This count covers only what the weight-holding layers hand on. It leaves out arrays made inside a single operation, such as attention's scores, which grow with the square of the prompt length. The output layer alone hands on 65,536 numbers per position, more than a quarter of the total, because it scores every vocabulary entry.

These are numbers handed from one layer to the next, not memory in use at one moment. Most activations are released as soon as the next layer has read them. Chapter 13 measures what a model actually needs in memory.

Running the model reads the parameters and never writes them. The next script copies all 132 arrays, generates three slogans, and compares:



TECHNICAL STUFF

LISTING 2-8 readonly.py

```
import torch
from load import tok, model

before = {n: p.clone() for n, p in model.named_parameters()}
Q = "Write a one-line slogan for a coffee shop."
msgs = [{"role": "user", "content": Q}]
ids = tok.apply_chat_template(
    msgs, add_generation_prompt=True,
    return_tensors="pt", tokenize=True)["input_ids"]
for seed in range(3):
    torch.manual_seed(seed)
    model.generate(ids, max_new_tokens=40)
same = sum(torch.equal(before[n], p)
           for n, p in model.named_parameters())
print(same, "of", len(before), "tensors unchanged")
```

OUTPUT OF LISTING 2-8

```
$ python3 readonly.py
132 of 132 tensors unchanged
```



Every number is exactly as it was. A model does not learn from the prompts you send it. Training changes parameters, and chapter 6 shows how. Running a model only makes activations, and sometimes a cache.

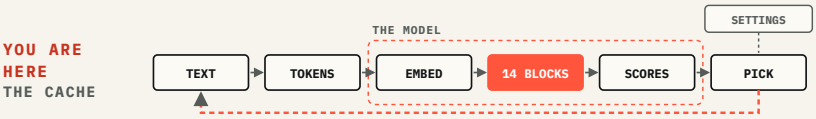


WATCH OUT

CHECK

Does a 352-token prompt use more parameters than a 6-token prompt? No. Both calls read the same 229,693,184 parameters. The 352-token prompt makes about 59 times more activations.

2.7 The cache: kept between steps



WATCH · 1:16

The cache

Chapter 1's loop sent the whole sequence through the model at every step. Step 0 processed 21 positions, step 1 processed 22, and so on, recomputing the same early positions again and again. The **cache** avoids that. After each call, the model keeps what later steps need, and the next call sends only the new token.

TERM **cache**
What the model keeps between steps of one reply, so earlier tokens are not recomputed. It belongs to one conversation.

LISTING 2-9 cache.py

```
import torch
from load import tok, model

Q = "What is the capital of France? Answer in one sentence."
msgs = [{"role": "user", "content": Q}]
ids = tok.apply_chat_template(
    msgs, add_generation_prompt=True,
    return_tensors="pt", tokenize=True)["input_ids"]
cache, new = None, ids                    # first call: the prompt
for step in range(40):
    with torch.no_grad():
        out = model(new, past_key_values=cache, use_cache=True)
    cache = out.past_key_values           # kept for the next call
    nxt = int(out.logits[0, -1].argmax())
    keys = cache.layers[2].keys           # attention block 2
    conv = cache.layers[0].conv_states[0]  # conv block 0
    print(f"{step} in {new.shape[1]:>2} {tok.decode([nxt])!r:12}"
          f" keys {tuple(keys.shape)} conv {tuple(conv.shape)}")
    if nxt == 7:
        break
    new = torch.tensor([[nxt]])           # next call: one token
```



TECHNICAL STUFF

OUTPUT OF LISTING 2-9

```
$ python3 cache.py
0 in 21 'The'         keys (1, 8, 21, 64) conv (1, 1024, 3)
1 in  1 ' capital'    keys (1, 8, 22, 64) conv (1, 1024, 3)
2 in  1 ' of'         keys (1, 8, 23, 64) conv (1, 1024, 3)
3 in  1 ' France'     keys (1, 8, 24, 64) conv (1, 1024, 3)
4 in  1 ' is'         keys (1, 8, 25, 64) conv (1, 1024, 3)
5 in  1 ' Paris'      keys (1, 8, 26, 64) conv (1, 1024, 3)
6 in  1 '.'           keys (1, 8, 27, 64) conv (1, 1024, 3)
7 in  1 '<|im_end|>' keys (1, 8, 28, 64) conv (1, 1024, 3)
```

The reply is the same eight tokens as chapter 1's loop, `The capital of France is Paris.` and the end-of-turn token. The difference is the input column. The first call sends the 21-token prompt. Every later call sends one token.

The two kinds of block keep different things:

TERM **keys and values**
The arrays attention looks back at. The cache keeps them for every earlier position.

- Each **attention block** keeps the **keys and values**, the arrays attention looks back at, for every position so far. Name the axes of `(1, 8, 21, 64)`: batch, 8 key-value groups, 21 positions, 64 numbers per group. The position axis grows by one each step, to 28 by the end. The cache grows with the conversation.

TERM **conv state**
What a conv block keeps between steps: the last three positions of the numbers its filter reads.

- Each **conv block** keeps its **conv state**: the last three positions of the numbers its filter reads. Name the axes of `(1, 1024, 3)`: batch, 1,024 numbers, 3 positions. Here the position axis comes last, and it never grows. Each step drops the oldest position and adds the newest.

That second point is the reason to name axes instead of assuming them. In the keys, the positions come before the 64 numbers of each group. In the conv state, the 1,024 numbers come first and the 3 positions last. Same idea, different order.

TERM **state**
Data the model carries from one call to the next, like the cache.

The cache is **state**: data the model carries from one call to the next. It is not parameters. It starts empty, it is built from activations, and it changes every step. It also saves real work. We ran both loops for 64 steps from this prompt without stopping at the end token. The cached loop processed 84 positions, the

recompute loop processed 3,360, and both produced the same 64 tokens. How much time that saves depends on the runtime and the hardware. Chapter 9 covers the loop with its cache, and chapter 13 covers the cost.

The cache grows with every token. Each attention block keeps 2 × 8 × 64 = 1,024 numbers per position, so the six attention blocks keep 6,144 numbers per position. The eight conv blocks keep 24,576 numbers in total, however long the conversation runs. We calculated that at 10,000 positions, the cache would hold about 61 million numbers, about a quarter as many as the parameters.

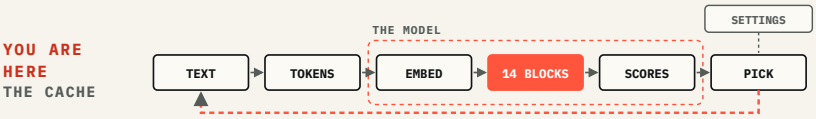
CHECK

After 28 positions, the keys in block 2 have shape `(1, 8, 28, 64)`. What shape will they have after 100 positions, and what about the conv state? The keys will be `(1, 8, 100, 64)`. The conv state stays `(1, 1024, 3)`.



QUICK CHECK

2.8 A cache belongs to one conversation



WATCH · 1:04



The wrong cache

The cache stands in for every earlier position. The model never sees those tokens again, only what the cache kept from them. So the cache must come from the same conversation as the new token. Here is what happens when it does not:



TECHNICAL STUFF

LISTING 2-10 stale.py

```
import torch
from load import tok, model

def ids(text):
    return tok(text, return_tensors="pt").input_ids
def top(out):
    return tok.decode([int(out.logits[0, -1].argmax())])

fr = ids("The capital of France is")
es = ids("The capital of Spain is")
with torch.no_grad():
    print("Spain, whole prompt:  ", repr(top(model(es))))
    cache = model(fr[:, :-1], use_cache=True).past_key_values
    out = model(es[:, -1:], past_key_values=cache)
    print("Spain, France's cache:", repr(top(out)))
```

OUTPUT OF LISTING 2-10

```
$ python3 stale.py
Spain, whole prompt:   ' Madrid'
Spain, France's cache: ' Paris'
```



WATCH IT GO WRONG

The two prompts are both six tokens long, and they differ only at position 4, `France` versus `Spain`. The script builds a cache from the first five tokens of the France prompt. Then it sends the last token of the Spain prompt, `is`, with that cache. The model answers `Paris`, with exactly the probabilities it gives the France prompt. As far as the model can tell, it was asked about France.

This is the kind of bug a runtime can have. It might reuse a cache because the earlier positions have the same count, without checking that they hold the same tokens. Or it might hand one conversation's cache to another. Either way, the model computes on text nobody sent in this conversation. The parameters are fine, and the model did exactly its job.

A cache is part of the input. Start each conversation with an empty cache, and reuse one only for the exact tokens it was built from.

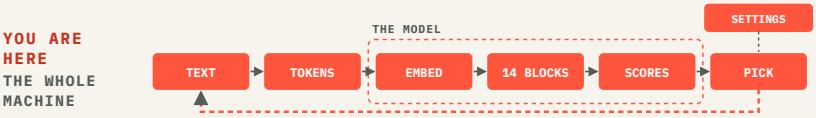
CHECK

Two users chat with the same loaded model at the same time. What can they share, and what must each have alone? They can share the parameters, which are only read. Each needs a separate cache, because a cache holds one conversation's earlier tokens.



QUICK CHECK

2.9 Sorting any array



WATCH · 1:20

Sort any array

Every array the model learns, makes during a call, or keeps for a conversation fits one of the kinds in this table. The config is listed too, because it fixes the shapes. Three arrays sit outside the table: the token IDs, which are the input the tokenizer hands the model, and two small lists of 32 numbers that the model computes from the config when it loads, for marking positions.

parameter	EXAMPLE `layers.0.feed_forward.w1.weight`		SHAPE (2560, 1024)	
	MADE BY training, then loaded from the weights file	LIVES while the model is loaded	CHANGES WHILE RUNNING? never	
activation	EXAMPLE block 5's output for the France prompt		SHAPE (1, 6, 1024)	
	MADE BY one call	LIVES until the next layer has used it	CHANGES WHILE RUNNING? made fresh each call	

cache	EXAMPLE block 2's keys		SHAPE (1, 8, n, 64)
	MADE BY the loop, from activations	LIVES one conversation	CHANGES WHILE RUNNING? grows one position per step
cache	EXAMPLE block 0's conv state		SHAPE (1, 1024, 3)
	MADE BY the loop, from activations	LIVES one conversation	CHANGES WHILE RUNNING? its three positions roll forward each step
config	EXAMPLE `hidden_size`		SHAPE one number, not an array
	MADE BY the publisher	LIVES ships with the model	CHANGES WHILE RUNNING? never

Three questions sort any array:



REMEMBER

1. **What are its axes?** Name each one before you index it.
2. **Who made it?** Training made the parameters. One call made the activations. The loop made the cache. The publisher wrote the config.
3. **How long does it live?** While the model is loaded, for one call, or for one conversation.

These questions also find the bugs in this chapter. The wrong-axis slice failed question 1. The stale cache failed question 3: it was used outside the conversation it belongs to. The double count needs one more check, for parameters: are two names the same array?

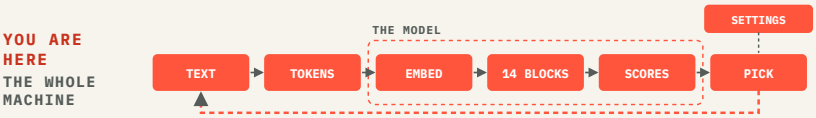


QUICK CHECK

CHECK

A runtime holds an array of shape `(1, 8, 412, 64)` between calls. Which kind is it, and what does 412 tell you? It is a cache: keys or values from an attention block. 412 is the number of positions the conversation has reached so far.

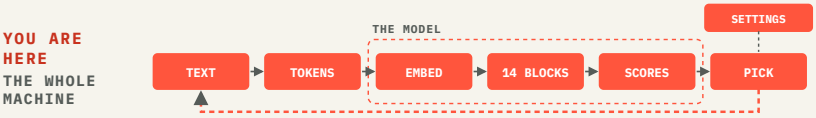
2.10 The map



Each part of this chapter opens up fully later in the book:

PART	OPENS IN
What multiplying by a (2560, 1024) weight matrix computes	**B1C03**
How each number is stored in two bytes	**B1C04**
How training sets the parameters	**B1C06**
Queries, keys, values, heads, and norms	**B1C08**
The cache in the generation loop	**B1C09**
The conv block's three-position window	**B1C10**
Array names in the weights file	**B1C11**
Memory and time for parameters, activations, and the cache	**B1C13**

2.11 Put it together



Every value in a running model is an array with named axes. Logits are (batch, position, vocabulary). Embeddings and block outputs are (batch, position, width). Name the axes before you index, and compare the shape you got with the shape you meant. The parameters are 132 learned arrays, 229,693,184 numbers, and a few config values fix every shape. A weight matrix is stored as (numbers out, numbers in). The output layer shares the embedding table, so count by storage,



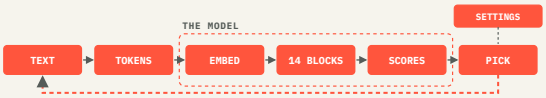
not by name. Activations are made for each call and grow with the batch and the positions. Running the model never changes a parameter. The cache keeps keys and values that grow by one position per step in the attention blocks, and a fixed three-position state in the conv blocks. The cache belongs to one conversation, and a stale one sends the model text from a different conversation.

The next chapter opens the weight matrices. It shows what multiplying a token's 1,024 numbers by a (2560, 1024) matrix computes, and why the shapes have to line up.

Further reading

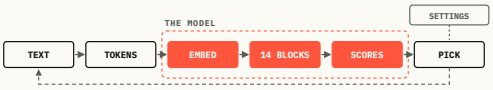
- PyTorch documentation: `torch.Tensor`, `torch.nn.Module` (`named_parameters`, `state_dict`), and `torch.nn.Linear`.
- Hugging Face Transformers documentation: "Caching" and the LFM2 model page.

Cheat Sheet

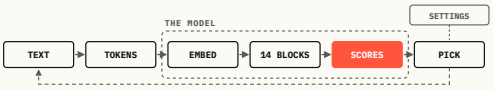


THE MACHINE · WHERE EACH PART IS TAUGHT

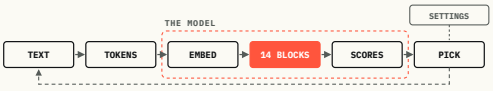
2.1 Every value has a shape



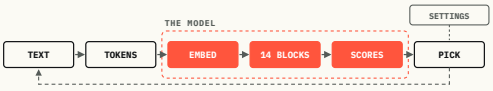
2.2 The wrong axis



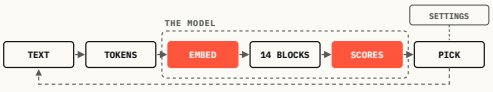
2.3 Reading a parameter's name



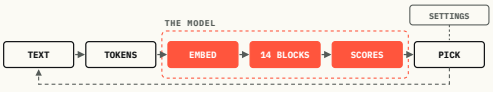
2.4 The config sets every shape



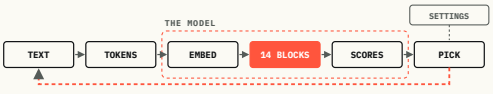
2.5 Counting the parameters once



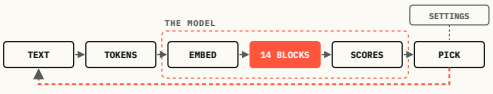
2.6 Activations: made for one call



2.7 The cache: kept between steps



2.8 A cache belongs to one conversation



TERMS · 19 IN THIS CHAPTER

tensor An array of numbers with a shape. Each position in the shape is an axis.

shape The list of axis sizes, in order. (2, 6, 1024) has three axes.

parameters The model's learned numbers, also called weights. Training sets them. Running the model only reads them.

activation An array the model computes during one call, like a block's output. Made fresh each call, then dropped.

cache What the model keeps between steps of one reply, so earlier tokens are not recomputed. It belongs to one conversation.

config The small file of numbers, like the width and the number of blocks, that fixes every array's shape.

axis One direction you can index along, like batch, position, or width. Name it before you use it.

batch Several independent prompts stacked into one array, so one call handles them all.

position Where a token sits in the sequence: 0 for the first token, 1 for the next.

width How many numbers describe one token inside the model: 1,024 here.

vocabulary Every token the model can score: 65,536 of them for this model.

block One repeated stage that reworks every token's numbers. This model stacks 14 of them. The code calls them layers.

weight matrix A parameter with two axes, stored as (numbers out, numbers in).

attention A mixer that lets each token pull information from every earlier token, weighted by content.

feed-forward The part of a block that reworks each token's numbers on its own, widening them and narrowing back.

keys and values The arrays attention looks back at. The cache keeps them for every earlier position.

conv state What a conv block keeps between steps: the last three positions of the numbers its filter reads.

loop Call the model, pick one token, append it to the input, and call again, until a stop token appears.

state Data the model carries from one call to the next, like the cache.